BUGLEX — Replication Guide

Siddharth Shringarpure

1 Purpose

This document explains how to reproduce the main experimental results reported for the bug report classification tool. The goal is to reproduce the saved CSV outputs, report tables, and report figures from the repository rather than to re-implement the pipeline from scratch.

2 Environment and Dependencies

Ensure your system meets the dependencies and environment requirements outlined in `requirements.pdf`. Initialise the project with:

```
uv sync
```

3 Reproducing the Main Results

3.1 Default Reported Results

To reproduce the main reported experiment outputs and regenerate the default figures, run:

```
uv run python -m src.run_experiments --with-plots
```

This command:

- evaluates all main models on all five datasets
- runs the 768-dimensional main comparison
- runs the 512, 256, 128, and 64 dimensional embedding ablation
- automatically performs statistical tests (Wilcoxon and Friedman)
- saves summary and statistical CSV outputs under `results/`
- regenerates the default figures in `results/figures/`

Notable reported files include:

- `results/summary_768.csv`
- `results/wilcoxon_summary_768.csv`
- `results/friedman_main_models_768.csv`
- `results/embedding_ablation_summary_512_256_128_64.csv`
- `results/main_table_macro_f1.csv`
- `results/figures/hybrid_ablation.png`

3.2 Preprocessing Ablation

To reproduce all preprocessing ablations as well:

```
uv run python -m src.run_experiments --all-preprocessing --with-plots
```

This repeats the experiment workflow for the default mode and the four non-default preprocessing modes. Their outputs are saved to mode-specific file names such as:

- `results/summary_768_lemmatize.csv`
- `results/summary_768_stopwords_all.csv`
- `results/summary_768_stopwords_keep_negation.csv`
- `results/summary_768_stopwords_keep_negation_plus_lemmatize.csv`

4 Protocol Being Reproduced

The key evaluation settings are:

- five datasets: Caffe, Incubator-MXNet, Keras, PyTorch, and TensorFlow
- 30 paired runs with seeds 0 to 29
- 70/30 stratified train-test split
- macro-F1 as the primary metric
- TF-IDF fit on the training split only
- one-sided paired Wilcoxon test for Hybrid LogReg versus the baseline
- supplementary Friedman test across the four main models

These settings are also summarised in `results/run_notes.txt` and the mode-specific `run_notes_*` files.

5 Cross-Platform Reproducibility

Results were produced on macOS 26.4.1 (Apple Silicon). Running on a different operating system or CPU architecture (eg: Linux-based x86-64 distributions) may yield slightly different numerical outputs, even with identical seeds and dependency versions. This may be caused by platform-level differences in dependency implementations of features, such as stratified sampling and tie-breaking. However, the differences are typically small and should not affect the relative ranking or ordering of models, nor the conclusions drawn from statistical tests.

6 Independent Verification via Continuous Integration

The repository workflows run the pipeline on a clean Ubuntu environment hosted by GitHub, independently of the machine used to produce the reported results. A smoke test runs on pushes to the repository, which installs all dependencies from scratch, confirms the embedding model loads with the correct output shape, and runs the TF-IDF baseline on the `caffe` dataset end-to-end. A second workflow, triggered manually, runs the full experiment pipeline. Passing these checks on a separate machine provides some independent confidence that the results are not specific to the development environment.

7 Caching and Expected Behaviour

Sentence embeddings are cached in `results/embeddings/`; this is expected and desirable for replication, as the reported runtime tables are cached model-stage costs rather than one-off embedding-construction costs. The embedding caches are named to distinguish them by dataset, dimension, model, and preprocessing mode, so a rerun might not require manual deletion of cached files.

8 Rebuilding the Report

After regenerating the results, rebuild the report tables and PDF with:

```
uv run python3.13 src/tools/build_docs.py --report-only
```

This command regenerates the LaTeX table fragments from the current `results/` CSV files and recompiles `docs/report/report.tex`.

9 Expected Final Artefacts

After a successful reproduction run, the repository should contain:

- the regenerated CSV outputs under `results/`
- the regenerated figures under `results/figures/`
- the rebuilt report PDF under `docs/report/report.pdf`
- root-level copies of all four PDFs: `report.pdf`, `requirements.pdf`, `manual.pdf`, and `replication.pdf`

10 Artefact Location

`replication.pdf` is placed in the repository root. The source for this PDF is stored in `docs/replication/`.